



UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería Electrónica, Robótica y Mecatrónica

Memoria de Práctica

Lab-MR5

Planificación de caminos II

Algoritmo A con heurística euclídea consistente*

Ampliación de Robótica

Saúl Gutiérrez Rodríguez

Pablo Haro García

Mario Merino Prado

Repositorio: github.com/marioomerino24/ampliacion_robotica

Entorno: ROS 2 Humble · CoppeliaSim · Pioneer 3-DX

Índice

1. Introducción	3
1.1. Objetivo de la práctica	3
1.2. Descripción del problema	3
2. Fundamentos teóricos	3
2.1. Algoritmo A*	3
2.2. Admisibilidad y consistencia	3
2.3. Heurística euclídea consistente	3
3. Diseño e implementación	4
3.1. Interfaz de la función	4
3.2. Validaciones de entrada	4
3.3. Estructuras del algoritmo	4
3.4. Selección del nodo a expandir	4
3.5. Relajación y reconstrucción	5
4. Resultados experimentales	5
4.1. Construcción del grafo euclídeo	5
4.2. Comparativa A* vs. Dijkstra	5
4.3. Casos límite verificados	5
5. Discusión	5
5.1. Por qué construir C^{eucl}	5
5.2. Alternativas a la construcción de C^{eucl}	6
5.3. Complejidad y rendimiento	6
5.4. Limitaciones	6
6. Conclusiones	6

1. Introducción

1.1. Objetivo de la práctica

El objetivo es implementar el algoritmo **A*** para calcular la ruta de coste mínimo entre dos nodos de un grafo topológico, incorporando una **heurística consistente** que permita reducir el número de nodos explorados respecto a Dijkstra (Lab-MR4) sin perder optimalidad.

1.2. Descripción del problema

La formulación del grafo es la misma que en Lab-MR4: matriz $G \in \mathbb{R}^{N \times N}$ con $G(i, j) > 0$ indicando la existencia del arco $i \rightarrow j$ y su peso, y dos nodos s (origen) y t (destino). La diferencia es que se dispone, además, de las **coordenadas geométricas** de los nodos en el espacio (matriz `nodos` con x e y por columnas), lo que permite definir una heurística admisible y consistente basada en la distancia euclídea.

2. Fundamentos teóricos

2.1. Algoritmo A*

A* expande nodos siguiendo el orden de la función de evaluación

$$f(v) = g(v) + h(v), \quad (1)$$

donde:

- $g(v)$ es el coste real acumulado desde el origen s hasta v ,
- $h(v)$ es una estimación heurística del coste de v a t .

Mantiene un conjunto **abierto** (nodos generados pero no expandidos) y un conjunto **cerrado** (nodos ya expandidos). En cada iteración extrae del abierto el nodo de menor f , lo cierra y relaja sus aristas salientes. Cuando se cierra el destino, la búsqueda termina.

2.2. Admisibilidad y consistencia

Una heurística h es:

- **admisible** si $h(v) \leq h^*(v)$, donde $h^*(v)$ es el coste real mínimo hasta el destino. Garantiza optimalidad.
- **consistente** (o monótona) si $h(u) \leq G(u, v) + h(v)$ para todo arco $u \rightarrow v$. La consistencia implica admisibilidad y, además, garantiza que un nodo no necesita reaperturarse una vez cerrado.

2.3. Heurística euclídea consistente

La heurística empleada es la distancia euclídea entre el nodo y el destino:

$$h(i) = \|p_i - p_t\|_2, \quad (2)$$

con $p_i = (x_i, y_i)$. Para que esta heurística sea **consistente** respecto a la matriz de costes utilizada, los pesos deben ser **compatibles** con la métrica euclídea, es decir, satisfacer $G(u, v) \geq \|p_u - p_v\|_2$ para todo arco. Cuando la matriz original del enunciado no cumple esta propiedad (porque sus pesos son arbitrarios), se construye una matriz alternativa:

$$C_{ij}^{\text{eucl}} = \begin{cases} \|p_i - p_j\|_2 & \text{si existe arco } (i, j), \\ 0 & \text{en caso contrario,} \end{cases} \quad (3)$$

manteniendo la **misma topología** pero con costes que coinciden con la métrica de la heurística. Así, A^* sobre C^{eucl} con h euclídea es óptimo y consistente.

3. Diseño e implementación

3.1. Interfaz de la función

La función se ha implementado en MATLAB con la signatura:

```
[coste, ruta] = astar(G, nodos, origen, destino)
```

donde `nodos` es una matriz $N \times 2$ (o $N \times M$ con x, y en las dos primeras columnas) con las coordenadas geométricas necesarias para la heurística.

3.2. Validaciones de entrada

Antes del bucle principal:

1. `G` cuadrada $N \times N$, con costes finitos y no negativos.
2. `nodos` con N filas y al menos 2 columnas.
3. `origen` y `destino` dentro del rango válido.
4. Si `origen == destino`, se devuelven `coste 0` y `ruta unipunto` sin entrar al bucle.

3.3. Estructuras del algoritmo

Se mantienen cuatro vectores de tamaño N :

- `g_score(v)`: mejor coste real conocido de s a v . Inicialmente ∞ , salvo `g_score(origen) = 0`.
- `f_score(v) = g_score(v) + h(v)`: criterio de expansión.
- `prev(v)`: predecesor en la ruta óptima parcial.
- `abierto(v)`, `cerrado(v)`: máscaras booleanas.

3.4. Selección del nodo a expandir

En cada iteración se busca el nodo abierto con menor f . En caso de **empate** en f , se rompe el desempate por menor h , lo que prioriza a los candidatos más cercanos al destino. Esta heurística de desempate suele reducir el número de nodos expandidos en grafos con costes equiparables.

3.5. Relajación y reconstrucción

Para cada vecino v con $G(u, v) > 0$ que no esté cerrado:

- se calcula $g_{\text{tent}} = g(u) + G(u, v)$,
- si v no está abierto, se añade,
- si $g_{\text{tent}} \geq g(v)$, se descarta el nuevo camino,
- en caso contrario, se actualizan $g(v)$, $f(v)$ y $\pi[v] = u$.

La reconstrucción de la ruta es idéntica a la de Dijkstra: se retrocede desde t por el vector de predecesores hasta llegar a s . Si en algún paso se encuentra un predecesor inválido se devuelve $\text{coste} = \text{Inf}$ y $\text{ruta} = []$.

4. Resultados experimentales

4.1. Construcción del grafo euclídeo

A partir de la matriz original del enunciado se construye C^{eucl} según la ecuación (3). La topología (qué pares (i, j) tienen arco) se preserva, y los pesos se sustituyen por distancias euclídeas. Esto se realiza una sola vez al principio del script de evaluación.

4.2. Comparativa A* vs. Dijkstra

La ruta resultante de A* sobre C^{eucl} y la de Dijkstra sobre la misma matriz coinciden, como debe ser por la consistencia de la heurística. La diferencia se observa en el número de nodos expandidos: A* descarta tempranamente las regiones del grafo cuyo f supera el coste óptimo conocido al destino.

[PENDIENTE: Captura comparativa de los nodos explorados por Dijkstra y por A* sobre el mismo grafo y mismo par origen-destino.]

Figura 1: Comparativa cualitativa de los nodos expandidos por Dijkstra y A*.

4.3. Casos límite verificados

- $\text{origen} == \text{destino}$: coste 0 y ruta de un solo nodo.
- Destino inalcanzable: $\text{coste} = \text{Inf}$, $\text{ruta} = []$.
- Heurística nula ($h \equiv 0$): A* degenera a Dijkstra. Sirve como prueba de regresión.

5. Discusión

5.1. Por qué construir C^{eucl}

Los pesos originales del enunciado no necesariamente coinciden con la distancia euclídea entre nodos. Aplicar A* directamente sobre la matriz original con h euclídea **podría violar la consistencia** (e incluso la admisibilidad), lo que se traduciría en rutas subóptimas. La construcción del grafo euclídeo es la forma más sencilla de garantizar las propiedades teóricas del algoritmo sobre estos datos.

5.2. Alternativas a la construcción de C^{eucl}

- Escalar la heurística para que $h(i) \leq G(u, v) + h(v)$ siempre. Garantiza admisibilidad pero no consistencia, y no aporta ventajas prácticas.
- Usar otra métrica (Manhattan, Chebyshev) si el grafo lo justifica. En este caso, la disposición libre de los nodos hace más natural la métrica euclídea.

5.3. Complejidad y rendimiento

La complejidad asintótica es la misma que la de Dijkstra ($O(N^2)$ con búsqueda lineal del mínimo), pero el número efectivo de nodos expandidos suele ser menor. La diferencia se nota especialmente cuando el destino está alejado del origen y el grafo tiene muchas ramificaciones poco prometedoras.

5.4. Limitaciones

- Requiere coordenadas geométricas de los nodos. No es aplicable a grafos puramente abstractos sin embedding.
- El uso de matrices completas y búsqueda lineal del mínimo no escala a grafos muy grandes; sería recomendable una cola de prioridad y representación dispersa.

6. Conclusiones

Se ha implementado A^* con heurística euclídea consistente sobre el grafo del enunciado, garantizando optimalidad mediante la construcción de una matriz de costes compatible con la métrica de la heurística. La interfaz es paralela a la de Lab-MR4 (*dijkstra*), lo que facilita comparaciones directas.

El algoritmo se reutiliza directamente en Lab-MR6, donde actúa como planificador global alternativo a Dijkstra para suministrar la secuencia de subobjetivos a la navegación reactiva por campos potenciales. La elección entre Dijkstra y A^* es transparente para el usuario y, en los grafos del enunciado, ambas rutas son equivalentes en coste y solo difieren en la fracción del grafo realmente explorada.